Blue Origin Panel Interview Preparation Guide

Data Acquisition and Controls Engineer II – New Glenn, R66427

Interview Study Notes

July 13, 2026

Abstract

This guide organizes technical and behavioral preparation for a Data Acquisition and Controls Engineer II panel interview supporting New Glenn component testing. The central theme is the ability to translate test objectives into safe, traceable, maintainable measurement-and-control systems: sensor selection, signal conditioning, acquisition, automated controls, interlocks, data validation, post-processing, and engineering review.

Core interview message

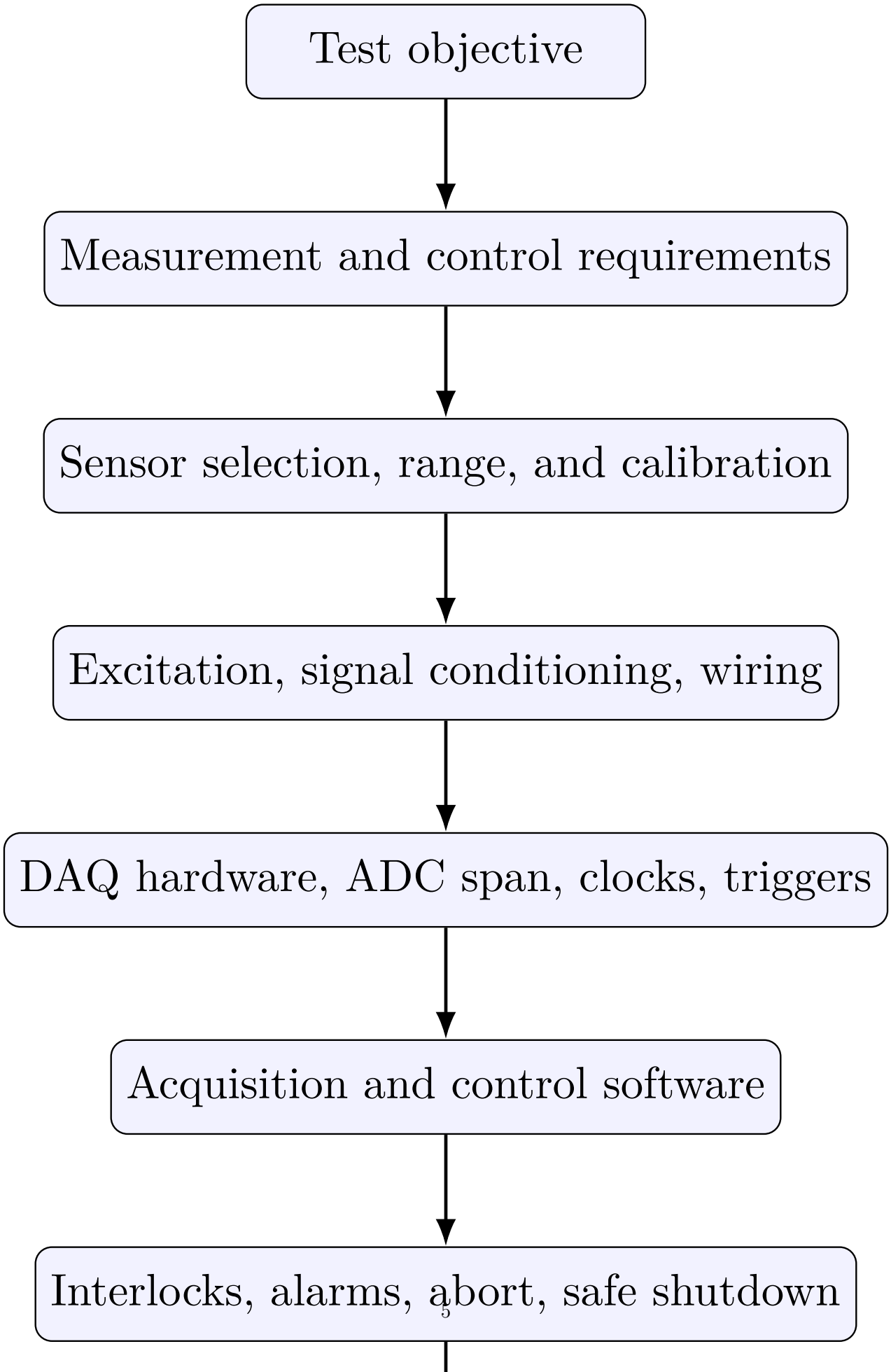
"I can translate test objectives into a safe, traceable, and maintainable measurement-and-control system–from sensor selection and signal conditioning through acquisition, automated control, data validation, post-processing, and engineering review."

Contents

1 Role Understanding

1.1 Core engineering mission

The role sits at the intersection of instrumentation, data acquisition, controls, electrical integration, test software, and engineering analysis. The panel is likely evaluating whether the candidate can build reliable test capability, not merely write code.


Test objective
Measurement and control requirements
Sensor selection, range, and calibration
Excitation, signal conditioning, wiring
DAQ hardware, ADC span, clocks, triggers
Acquisition and control software
Interlocks, alarms, abort, safe shutdown

1.2 What the panel is hiring you to do

- Convert test requirements into instrumentation and control plans.
- Select, integrate, calibrate, and validate sensors.
- Design or review signal-conditioning, grounding, shielding, wiring, and electrical interfaces.
- Configure DAQ hardware, clocks, triggers, ADC ranges, and acquisition software.
- Develop automated and semi-automated control logic.
- Ensure fail-safe operation, interlocks, alarms, and safe shutdown behavior.
- Store reliable, traceable, time-aligned data with useful metadata.
- Reduce, visualize, and review large datasets.
- Troubleshoot hardware/software problems under schedule pressure.
- Document systems so another engineer can operate and maintain them.

1.3 Engineer II expectations

At this level, expect questions that test independent execution, judgment, and cross-disciplinary troubleshooting. The panel will likely expect you to:

- Execute scoped work independently and identify unclear requirements.
- Make defensible technical decisions and explain tradeoffs.
- Troubleshoot across sensors, wiring, DAQ, software, controls, and data analysis.
- Escalate appropriately when safety, authority, or requirements are unclear.
- Communicate with test, mechanical, electrical, fluids, controls, and analysis engineers.
- Produce maintainable solutions rather than one-off scripts.
- Demonstrate safety judgment without exaggerating expertise.

2 Likely Panel Structure

The interview loop may include a hiring manager, DAQ/test software engineer, controls engineer, instrumentation or electrical engineer, test operations engineer, cross-functional stakeholder, and behavioral interviewer.

Category	What to prepare
Resume/project deep dive	Your individual contribution, architecture decisions, failures, validation, and measurable impact.
DAQ and instrumentation	Sensors, signal conditioning, sampling, aliasing, ADCs, clocks, triggers, uncertainty.
Controls/test automation	State machines, PID, interlocks, watchdogs, aborts, commissioning, deterministic timing.
Python/software	Modular design, hardware abstraction, logging, testing, concurrency, data persistence.

Electrical troubleshooting	Schematics, grounding, shielding, noise, return paths, isolation, terminal wiring.
Data analysis	Traceability, filtering, time alignment, pass/fail metrics, metadata, reports.
Leadership Principles	STAR stories tied to ownership, safety, operational excellence, trust, and backbone.
Mission motivation	Why Blue Origin, why New Glenn, why this specific end-to-end test role.

3 Sixty-to-Ninety Second Introduction

3.1 Recommended structure

1. Current technical identity.
2. Two or three capabilities relevant to DAQ, controls, test software, and analysis.
3. One representative accomplishment with measurable impact.
4. Why this position is the logical next step.
5. Why Blue Origin and New Glenn.

Model introduction

"I am an engineer focused on real-time simulation, controls, test automation, and data-driven system validation. In my current work, I support real-time simulation environments where software, instrumentation, timing, and system behavior must remain synchronized and reliable.
A representative example is a project where I developed or improved an automated test and analysis workflow for [system]. I integrated [hardware/software], implemented validation and fault detection, and improved [test time, reliability, repeatability, or data quality] by [measured result].
This Blue Origin position is attractive because it combines the areas I most enjoy: instrumentation, Python development, real-time systems, controls, test execution, and troubleshooting. I am motivated by the direct connection between test-data integrity and safe, repeatable flight hardware, and I would like to bring my real-time simulation and automation experience into component testing for New Glenn."

4 DAQ Fundamentals

4.1 Sensor selection

When selecting a sensor, discuss:

- Range, proof pressure, burst pressure, over-range behavior, and fault margin.
- Accuracy, precision, repeatability, linearity, hysteresis, drift, and total uncertainty.
- Sensitivity, resolution, signal-to-noise ratio, and usable span.
- Bandwidth, dynamic response, mounting effects, and sensor loading.
- Environmental rating, fluid compatibility, calibration, and failure modes.

Range tradeoff

A very large range may survive the test but provide poor useful resolution. A tight range may improve resolution but fail to capture transients or fault conditions. Selection should be tied to measurement objective and credible off-nominal behavior.

4.2 Important sensor types

Sensor	Topics to know
Pressure transducer	Absolute, gauge, sealed gauge, differential; voltage and current outputs; proof/burst pressure; fluid compatibility; temperature effects; static versus dynamic measurement; tubing effects.
Thermocouple	Seebeck effect; type selection; cold-junction compensation; low-level voltage; extension wire compatibility; grounded versus ungrounded junctions; noise; response time; open detection.
RTD	Two-, three-, four-wire measurements; lead resistance; excitation current; self-heating; accuracy and stability; differences from thermocouples.
Load cell/strain gauge	Wheatstone bridges; quarter/half/full bridge; excitation; mV/V sensitivity; shunt calibration; temperature compensation; preload; load path; cable shielding; zero balance; creep.

4.3 Sensor-selection question

Question. You need to measure pressure expected near 2,000 psi, but short transients could reach 2,600 psi. How would you select the transducer?

Strong answer. First clarify required accuracy, frequency content, fluid compatibility, temperature, proof pressure, burst pressure, and whether the desired pressure is absolute, gauge, sealed gauge, or differential. Do not select a 2,000 psi device just because that is the nominal operating point. Choose a range that safely captures credible transients while preserving useful resolution, possibly around 3,000 psi depending on uncertainty and safety margins. Then evaluate total measurement uncertainty: sensor accuracy, temperature effects, signal conditioning, ADC accuracy, calibration, mounting, and dynamic pressure-line effects. Define over-range behavior, calibration requirements, and reasonableness checks against related measurements.

Question. What if the pressure has a high-frequency oscillation?

Strong answer. Evaluate transducer frequency response and the dynamics introduced by tubing, cavities, and mounting. Tubing can damp or resonate, so a high DAQ sample rate alone does not ensure faithful measurement. Clarify whether the objective is mean pressure, transient pressure, or oscillation characterization because each can drive different sensor, mounting, and sampling choices.

5 Signal Conditioning, ADCs, and Sampling

5.1 Signal conditioning

Prepare to explain amplification, attenuation, anti-alias filtering, isolation, sensor excitation, bridge completion, cold-junction compensation, linearization, current-to-voltage conversion, terminal configuration,

common-mode voltage, common-mode rejection, input impedance, ground loops, shielding, and cable routing.

Examples

- A load cell with 2 mV/V sensitivity and 10 V excitation produces only 20 mV at rated load, so it normally requires low-noise differential measurement and appropriate gain.
- A 4–20 mA pressure transmitter can be useful in long-cable or noisy environments because current loops are robust and can indicate some open-circuit or under-range conditions.

5.2 Sampling and aliasing

Nyquist says the sample rate must exceed twice the highest frequency component of interest. Practical engineering requires more: anti-alias filters are not ideal, transients may require more samples per event, and synchronized channels may impose timing constraints. Consider:

- Highest physical signal frequency of interest and unwanted high-frequency content.
- Analog anti-alias filter cutoff, order, roll-off, and phase behavior.
- Desired time-domain fidelity and channel synchronization.
- Data volume, storage, network bandwidth, and control-loop timing.
- Hardware-timed versus software-timed acquisition.

Question. A sensor contains meaningful content up to 100 Hz. What sampling rate would you use?

Strong answer. The theoretical lower limit is greater than 200 samples/s, but the rate should not be selected from Nyquist alone. Review anti-alias filter cutoff and roll-off, transient fidelity, time-alignment needs, control requirements, and storage constraints. A rate such as 1 kS/s may be reasonable for good time-domain representation, provided unwanted content above Nyquist is attenuated before digitization.

Weak answer to avoid

"Exactly 200 Hz because Nyquist says twice the frequency." This ignores realizable filters, guard band, timing fidelity, and unwanted high-frequency content.

5.3 ADC fundamentals

Question. What determines the smallest voltage change an ADC can represent?

Strong answer. The ideal quantization step is the selected input span divided by the number of ADC codes. For an N-bit ADC, there are nominally 2^N codes. Effective resolution is also affected by noise, gain error, offset, nonlinearity, reference stability, front-end configuration, and sensor usable span. Distinguish ideal bit resolution from effective number of bits and overall measurement uncertainty.

Question. What is the difference between accuracy and resolution?

Strong answer. Resolution is the smallest representable increment. Accuracy is closeness to the true value. A system can display many digits while still being inaccurate due to offset, drift, calibration error, sensor error, electrical noise, or poor installation.

6 Grounding, Shielding, and Noise Troubleshooting

6.1 Topics to review

Ground loops, common-mode noise, differential inputs, single-point grounding, shield termination, twisted pairs, sensor/power separation, electromagnetic interference, isolation, cable length/capacitance, chassis ground versus signal common, 50/60 Hz pickup, switching noise, and thermocouple grounding problems.

Do not over-generalize

"Ground the shield at one end" is not a universal law. The correct shield strategy depends on frequency, system architecture, isolation, chassis bonding, safety grounding, and manufacturer guidance.

Question. A pressure measurement is stable when the pump is off but noisy when the pump starts. How would you troubleshoot it?

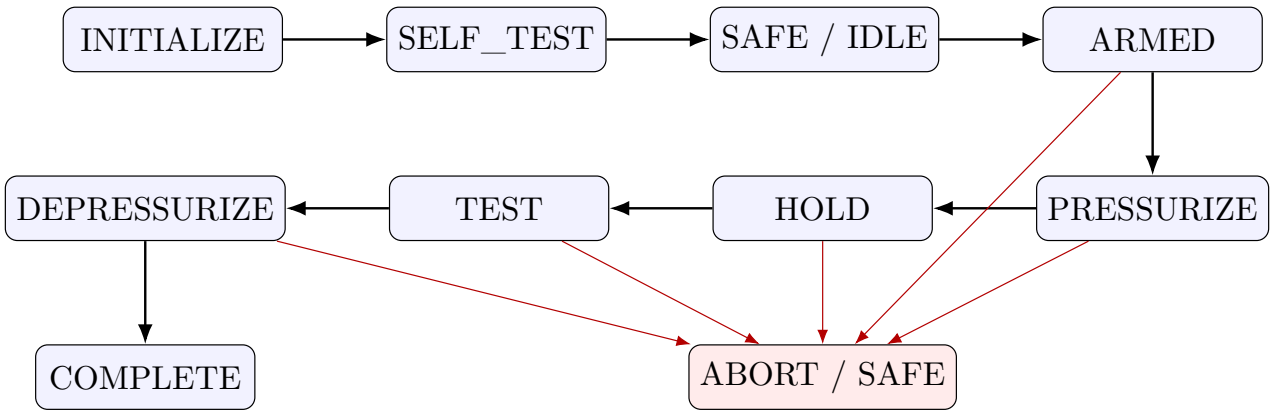
Strong answer. First keep the system safe and avoid changing multiple variables at once. Characterize the noise: amplitude, frequency, affected channels, and correlation with pump speed, switching, or flow conditions. Separate possible causes into real pressure pulsation, electrical interference, grounding, mechanical vibration, signal-conditioning saturation, and software configuration. Compare nearby sensors, inspect raw data rather than only filtered trends, check shield and ground continuity, review cable routing near motor power, and measure the signal at successive points in the chain. Substitute a known signal or simulator at the DAQ input. If clean, focus upstream; if noisy, focus on conditioning, cabling, grounding, or DAQ setup. Document each change. Do not use a digital filter to hide an unresolved physical or electrical issue. Also verify that "noise" is not legitimate pressure pulsation important to hardware safety.

7 Controls and Automated Test Operations

7.1 Control-system fundamentals

Review open-loop versus closed-loop control, PID, stability, overshoot, settling time, sensor delay, actuator saturation, integrator windup, rate limiting, setpoint ramping, discrete-time implementation, control-loop frequency, state machines, fault states, interlocks, permissives, watchdogs, emergency shutdown, manual/automatic modes, bumpless transfer, and determinism.

7.2 State-machine design



For every state define entry conditions, permissives, commands, expected sensor responses, timeouts, exit conditions, fault triggers, safe-state actions, operator authority, and logged events.

Question. Design the control architecture for an automated pressure test.

Strong answer. Begin by clarifying hardware, pressure range, test medium, credible hazards, ramp profile, hold duration, leakage limits, abort criteria, and safety criticality. Implement an explicit state machine rather than an unstructured sequence. Before arming, verify sensor health, communications, valve positions, pressure boundaries, emergency-stop status, and environmental conditions. During pressurization, apply a controlled ramp with high-pressure, high-rate-of-rise, sensor-disagreement, timeout, valve-response, and communication-loss checks. Safety-critical shutdown should not depend solely on a non-deterministic user interface. Where required by hazard analysis, protection belongs in independent hardware or a dedicated safety layer. Timestamp every command, state transition, interlock, and operator action. Define safe behavior for power loss, communication loss, invalid data, and software failure. Validate with simulation, dry runs, fault injection, and low-energy commissioning before full conditions.

Question. A pressure-control loop oscillates around the setpoint. What could cause it?

Strong answer. Possible causes include excessive proportional or integral gain, insufficient damping, sensor noise, process delay, actuator deadband, valve stiction, saturation, incorrect loop timing, unit errors, or operation away from the tuning point. Review trends for setpoint, measurement, error, controller output, valve feedback, and saturation. Verify timing and units before retuning. If the actuator is saturating, review anti-windup and command limits.

8 Test Safety and Fault Management

8.1 Safety hierarchy

Use layered protection:

1. Inherently safer design.
2. Physical limits and protective hardware.
3. Independent interlocks.
4. Control software protections.
5. Procedural controls.
6. Operator training and personal protective equipment.

Important safety statement

A Python exception handler is not a substitute for independent overpressure protection, emergency-stop chains, relief devices, or a safety-rated control layer when the hazard analysis requires them.

Question. What should the test stand do if communication with the controller is lost?

Strong answer. The correct behavior must come from the hazard analysis and safe-state definition. Do not assume holding the last command is safe. Outputs should transition predictably under communication loss, watchdog timeout, application failure, and power loss. Critical protection should remain effective even if the supervisory application is unavailable.

Question. You are close to a scheduled test, but an interlock is producing intermittent trips. What do you do?

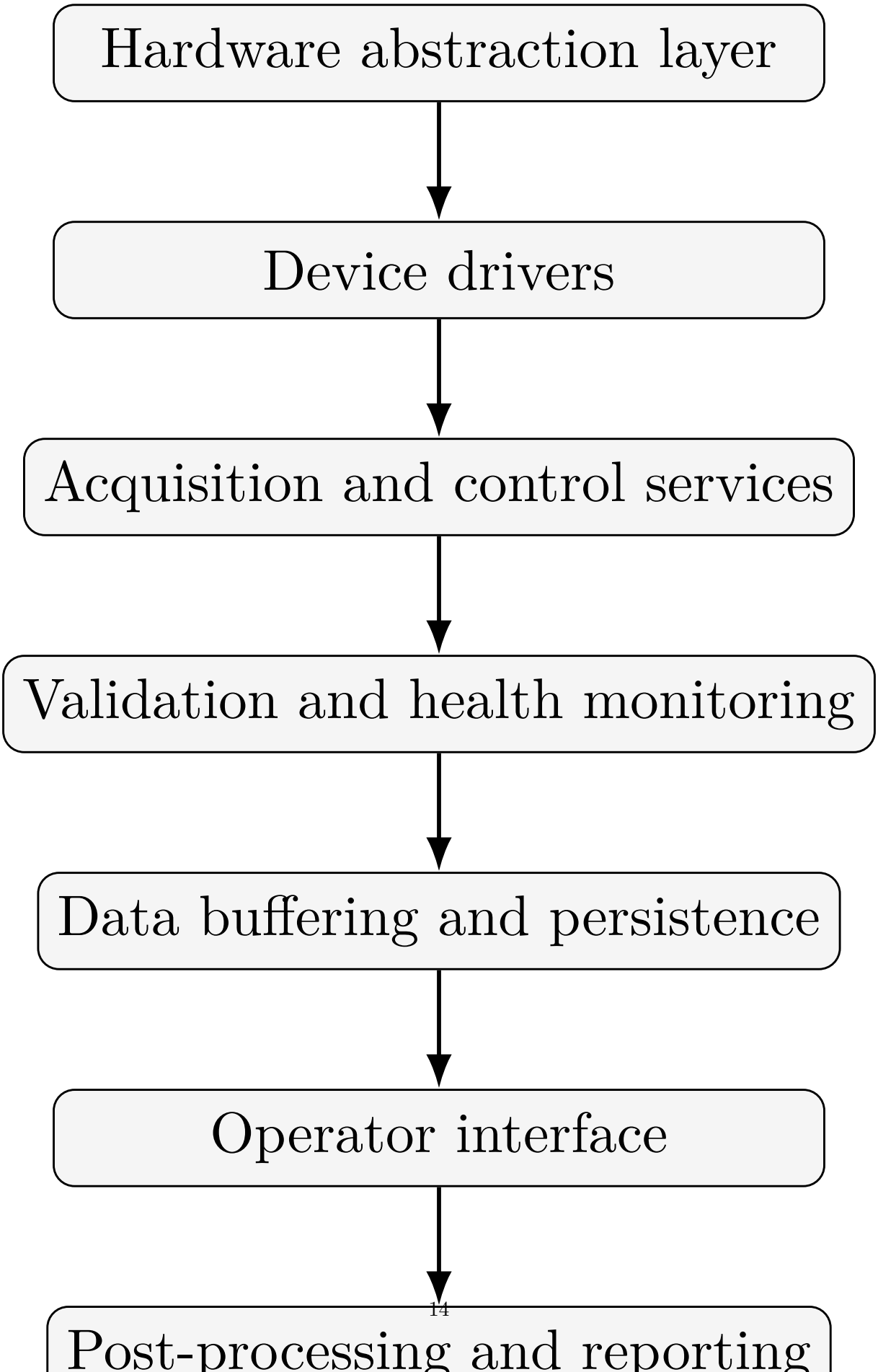
Strong answer. Do not bypass a safety interlock simply to preserve schedule. Pause, characterize the fault, and determine whether the trip is a real unsafe condition, instrumentation fault, configuration error, or interlock implementation issue. Engage responsible disciplines. A temporary configuration, if allowed at all, requires documented risk evaluation and authorization. Schedule pressure does not change physical risk.

9 Python and Software Engineering

9.1 Topics to prepare

Classes, modules, type hints, exceptions, context managers, logging, unit tests, mocking hardware interfaces, configuration management, concurrency, threads versus processes, queues, async I/O, NumPy, pandas, plotting, file formats, data validation, profiling, packaging, Git, code review, and continuous integration.

9.2 Recommended DAQ software architecture



Benefits: hardware can be mocked, UI failures need not stop acquisition, scaling and calibration are centralized, drivers can be replaced, and fault handling is easier to verify.

Question. How would you design Python software to acquire 100 channels continuously?

Strong answer. Start with sample rates, synchronization, latency, data volume, acceptable loss, and hardware-driver constraints. Separate acquisition from visualization and disk writing so slow UI or file operations cannot block time-critical reads. Acquisition should place timestamped blocks into bounded queues or ring buffers. A persistence worker writes chunked data and metadata; a display path decimates independently. Monitor queue depth, dropped samples, hardware status, timing errors, and disk capacity. Use versioned configuration files, structured logs, graceful shutdown, and automatic file finalization. Test with simulated devices, long-duration soak tests, injected failures, and known waveforms before commissioning with hardware.

Question. What happens if the disk writer falls behind?

Strong answer. Define behavior deliberately. Monitor buffer occupancy and warn early. Depending on test criticality, the system might abort safely before data loss, reduce nonessential display processing, or temporarily absorb backlog in a bounded buffer. Do not permit silent data loss.

10 NI Hardware, LabVIEW, and DAQmx

10.1 Concepts to review

NI CompactDAQ versus PXI/PXIe, multifunction DAQ devices, analog input/output, digital I/O, counter/timer channels, hardware-timed and software-timed acquisition, sample clocks, start triggers, reference triggers, module synchronization, buffered tasks, regeneration, DAQmx tasks/channels, terminal configuration, continuous versus finite sampling, LabVIEW producer-consumer architecture, LabVIEW state machines, Real-Time, and FPGA concepts.

Question. What is the advantage of hardware-timed acquisition?

Strong answer. Hardware timing uses DAQ timing resources rather than relying on operating-system scheduling for every sample. It provides more consistent sample intervals and better synchronization. The host application transfers blocks of samples while the hardware sample clock determines acquisition timing.

Question. How should you answer if your LabVIEW experience is limited?

Strong answer. Be direct: “My direct experience is stronger in Python and real-time simulation than in LabVIEW. However, I understand the core DAQ concepts–tasks, channels, clocks, triggers, buffering, scaling, and synchronized acquisition. I have also worked with modular software and state-machine architectures, so I would transfer those principles while learning Blue’s established design patterns and code-review expectations.”

11 Data Processing, Validation, and Visualization

11.1 Robust post-processing pipeline

A robust pipeline should:

- Read raw data without altering it and verify file integrity.
- Load calibration and configuration versions.
- Convert to engineering units and preserve raw samples.
- Handle invalid or missing samples explicitly.
- Align channels in time and detect the relevant test interval.
- Apply justified filtering with documented parameters.
- Calculate metrics, propagate quality flags, and generate plots/reports.
- Preserve traceability to raw data, procedure, configuration, and software revision.

Question. How would you determine whether a component passed a test?

Strong answer. Do not rely on a single opaque pass/fail calculation. Define acceptance criteria with the responsible engineering authority. Identify exact channels and time intervals, verify data quality, and compute each metric with units and uncertainty. The output should show result, limit, margin, quality status, and traceability to raw data, configuration, calibration, and analysis-code version. If required channels are invalid, the correct result may be "indeterminate" rather than pass or fail.

Question. When is filtering appropriate?

Strong answer. Filtering is appropriate when it follows from the physical measurement objective and has documented characteristics. Preserve raw data; specify filter type, order, cutoff, phase behavior, and edge handling. Do not choose a filter only because the plot looks cleaner.

Question. Two channels appear offset by 20 ms. What might cause it?

Strong answer. Possible causes include separate sample clocks, unsynchronized devices, software timestamps, channel multiplexing, filter group delay, transport buffering, different sensor response times, or timestamp interpretation errors. Establish whether the offset originates in physical sensors, acquisition hardware, communication, or post-processing before shifting data.

12 Databases and Data Architecture

12.1 Concepts to know

Raw waveform storage versus metadata storage, relational databases, time-series databases, HDF5, TDMS, Parquet, binary files, indexing, schema design, data retention, checksums, access control, configuration versioning, calibration traceability, test identifiers, searchability, backup, and recovery.

Question. How would you organize test data?

Strong answer. Separate immutable raw measurements from searchable test metadata. Each test should have a unique identifier linked to the unit under test, procedure version, software commit, DAQ configuration, channel map, calibration records, operator, timestamps, and environmental conditions. Large waveform data may belong in a structured file or object store, while a relational database indexes the test and metadata. The architecture should let an engineer reproduce how an engineering-unit value and final metric were generated.

13 Electrical Schematics and Wiring Diagrams

13.1 What to review

Power distribution, voltage rails, fuses, breakers, relays, contactors, grounding, terminal blocks, connector pinouts, cable and wire identifiers, analog and digital signal paths, normally open/closed contacts, interlocks, emergency-stop chains, shield termination, sensor excitation, return paths, and isolation boundaries.

Question. A sensor reads full scale regardless of applied input. How would you diagnose it?

Strong answer. Confirm the condition is safe and determine whether this is raw ADC saturation or a software-scaled full-scale result. Inspect channel type, terminal mode, input range, scaling, and excitation. Use the schematic to trace the signal from sensor to terminal block, conditioner, connector, and DAQ input. Measure known points with appropriate equipment. Inspect for open return, short to excitation, wrong pinout, loss of bridge completion, common-mode violations, and scaling errors. Apply a known simulator at the DAQ input to isolate sensor/wiring from the acquisition chain.

14 Test Stand Commissioning

14.1 Progressive commissioning sequence

1. Design, hazard, schematic, and configuration review.
2. Wiring inspection, continuity, isolation, and power-off verification.
3. Controlled power-up and power rail verification.
4. I/O checkout with known stimuli and sensor simulators.
5. Actuator command/feedback tests under controlled conditions.
6. Interlock, abort, permissive, and fault-injection verification.
7. Dry run, low-energy test, and incremental envelope expansion.
8. Readiness review, configuration baseline, and full test.

Question. How would you commission a new test stand?

Strong answer. Use a written commissioning plan with expected results and signoffs. Start with drawing and configuration review, then verify wiring, grounding, power, and protective devices before energization. Check every I/O channel with known stimuli and verify engineering-unit scaling end to end. For outputs, test commands under controlled conditions and confirm both command and independent feedback. Exercise permissives, aborts, communication failures, sensor failures, and power-loss behavior. Conduct dry runs and gradually increase test energy. Track discrepancies to closure. The released baseline should include hardware, software, channel map, calibration, and procedure versions.

15 Open-Ended System Design Exercise

15.1 Prompt

"Design a DAQ and control system for testing a pressurized component."

15.2 Step 1: Clarify requirements

Ask about the component, test objective, medium, normal and maximum pressures, temperatures, required dynamics, channel count, safety-critical measurements, controlled actions, timing accuracy, pass/fail criteria, safe state, redundancy, duration, and applicable standards.

15.3 Step 2: Define instrumentation

Possible measurements include inlet/outlet pressure, differential pressure, temperature, flow rate, load or strain, valve position, supply voltage/current, ambient conditions, and vibration if relevant.

15.4 Step 3: Build measurement chains

For each channel define:

quantity → sensor → excitation → conditioning → wiring/shielding → DAQ input → scaling → quality check → stor

15.5 Step 4: Define control architecture

Include state machine, permissives, interlocks, setpoint ramps, timeouts, watchdogs, abort logic, manual-mode restrictions, hardware safety layer, and operator interface.

15.6 Step 5: Define data architecture

Include hardware timestamps, shared clocks/triggers, channel map, calibration version, configuration version, raw and processed data, event log, operator log, software commit, and test article serial number.

15.7 Step 6: Define verification

Include unit tests, integration tests, sensor simulators, known electrical inputs, fault injection, latency tests, long-duration tests, data-recovery tests, safe-shutdown tests, and an end-to-end dry run.

16 Leadership Principles and STAR Preparation

16.1 Blue Origin principles to map

Prepare flexible STAR stories for:

- **Passion for Our Mission**
- **Customer Focus**
- **Deliver Results**
- **Embrace Team Blue**
- **Resourceful**
- **Bias for Action**
- **Ownership**

- **Operational Excellence**
- **Earn the Trust of Others**
- **Hire and Develop the Best**
- **Insist on the Highest Standards**
- **Technical Ambition and Simplicity**
- **Practice Humility**
- **Have Backbone; Disagree and Commit**

16.2 STAR structure

Part	Share	Content
Situation	10–15%	Give only context needed to understand the problem.
Task	10–15%	State your responsibility and desired outcome.
Action	55–65%	Explain what you personally did, decisions, alternatives, risks, collaboration, and validation.
Result	15–20%	Include measurable result, safety/quality impact, lesson learned, and lasting mechanism.

17 Behavioral Questions and Model Answers

17.1 Ownership

Question. Tell me about a problem outside your formal responsibility that you took ownership of.

Strong answer. Situation: repeated failures were treated as isolated software issues while ownership was split across teams. Task: your assigned area was narrower, but the interface problem threatened schedule and reliability. Action: you created an end-to-end signal map, gathered controls/hardware/test stakeholders, reproduced the issue, isolated root cause, documented assumptions, implemented the fix, and added an automated pre-test check. Result: failure rate dropped from [X] to [Y], the team recovered [time], and the diagnostic became standard workflow.

17.2 Operational Excellence

Question. Tell me about a recurring defect you eliminated.

Strong answer. A recurring data-quality issue was being handled by reruns. You analyzed history and found that a configuration, calibration, or timing condition was not validated before acquisition. You added an automated preflight check, configuration signature, and explicit failure message. The mechanism replaced "be careful" with prevention, reduced retests/manual effort by [amount], and made remaining failures easier to diagnose.

17.3 Highest Standards and safety

Question. Tell me about a time you stopped or delayed work because of a quality concern.

Strong answer. Before a test/release, you found a mismatch between configured limits and an approved requirement. Proceeding would meet schedule but create risk. You paused the activity, collected evidence, involved responsible stakeholders, found root cause, corrected the configuration, independently verified it, and resumed after review. The delay prevented invalid or unsafe testing and led to a future configuration-verification check.

17.4 Have Backbone; Disagree and Commit

Question. Tell me about a time you disagreed with a technical decision.

Strong answer. The team favored an approach that you believed introduced risk. You first understood the opposing argument, then presented evidence comparing reliability, timing, cost, safety, or maintainability. You proposed a small experiment to resolve uncertainty. Whether your approach or the other approach won, you committed to executing the final decision successfully.

17.5 Practice Humility

Question. Tell me about a mistake.

Strong answer. Use a real mistake, not a disguised strength. Explain the incorrect assumption, impact, containment, communication, correction, and mechanism added to prevent recurrence. End with a specific technical or communication lesson learned.

17.6 Deliver Results

Question. Tell me about a project with an aggressive deadline.

Strong answer. Reduce the work to critical deliverables, distinguish required functions from enhancements, attack high-risk interfaces first, and use short validation milestones. When setbacks occur, communicate early, adapt the approach, and deliver the result without waiving required safety or validation steps.

17.7 Resourceful

Question. Tell me about accomplishing something with limited tools or resources.

Strong answer. If hardware or test access was unavailable, describe building a simulator, replay tool, mock interface, or automated harness. State what it could and could not validate. Explain how it enabled early testing, reduced integration time, and became reusable.

17.8 Earn the Trust of Others

Question. Tell me about working with a difficult stakeholder.

Strong answer. Avoid labeling the person as difficult. Focus on conflicting objectives. Understand concerns, summarize agreement, make interfaces explicit, follow through on small commitments, share data openly, invite them into validation, and complete the result even if every detail was not agreed upon.

18 Failure and Troubleshooting Questions

18.1 Common follow-ups

Expect:

- What exactly did you do versus what did the team do?
- Why did you choose that approach, and what alternatives did you reject?
- What data supported your conclusion?
- How did you validate the fix and prevent recurrence?
- What was the final root cause?
- What would you do differently?
- Did anyone disagree?
- What was the measurable result?
- What safety consequences existed?
- How did you know the result was correct?

18.2 Troubleshooting framework

1. Make the condition safe.
2. Define the symptom precisely and establish what changed.
3. Reproduce the issue if safe and useful.
4. Separate hardware, software, configuration, and physical causes.
5. Change one variable at a time and instrument the diagnostic process.
6. Test the hypothesis and validate the fix under representative conditions.
7. Add a mechanism preventing recurrence.
8. Document the outcome.

19 Mission and Motivation

Question. Why Blue Origin?

Strong answer. "I am interested in Blue Origin because the mission connects long-term space access with engineering that must work safely and repeatedly. This role is especially compelling because test-system quality directly affects whether engineering teams can trust the evidence used to qualify hardware. My background in [real-time simulation, controls, automation, Python, or instrumentation] aligns with the full workflow described in the position–from acquiring synchronized data and automating test operations to troubleshooting integration issues and producing reliable engineering metrics. New Glenn presents the kind of complex, multidisciplinary environment where I want to grow."

Question. Why New Glenn?

Strong answer. "New Glenn interests me because reusability places a premium on consistent testing, traceable data, and understanding hardware across repeated operating cycles. A reusable vehicle requires not only successful operation but evidence that components and systems remain within acceptable margins. I am motivated by building test infrastructure that produces that evidence reliably."

Question. Why this specific position?

Strong answer. "This position does not isolate software from hardware. It combines sensor integration, electrical systems, DAQ architecture, controls, Python, data processing, and test execution. That end-to-end responsibility is attractive because I like understanding how a physical measurement becomes a trusted engineering decision."

20 Questions to Ask the Panel

20.1 Hiring manager

- What are the highest-priority test capabilities this engineer would support in the first six months?
- What differentiates an Engineer II who meets expectations from one who excels?
- Is the work primarily new test-system development, sustaining existing systems, or both?
- What are the team's largest technical bottlenecks today?
- How is ownership divided among test engineering, instrumentation, controls, software, and operations?
- What does commissioning and readiness review look like?

20.2 DAQ/software engineer

- What DAQ platforms and software architectures are most common?
- How do you manage channel configuration, calibration, and metadata?
- How are software releases validated before use on hardware?
- How do you simulate hardware interfaces during development?
- What time-synchronization challenges are most important?
- How do you balance reusable frameworks with test-specific requirements?

20.3 Controls/test-operations engineer

- How are safety-critical protections divided among hardware interlocks, real-time control, and supervisory software?
- What level of automation is typical during component testing?
- What are the most common causes of commissioning delays?
- How are off-nominal conditions and abort sequences validated?
- How much direct test execution and console support would this role perform?

Strong closing question

"Based on our discussion, is there any part of my experience or technical background that you would like me to clarify before we finish?"

21 One-Page Interview Summary Template

Heading

Data Acquisition and Controls Engineer II – New Glenn

Understanding of the role

This role develops and commissions component test systems for New Glenn, integrating sensor instrumentation, DAQ hardware and software, automated controls, data management, and post-processing. The central responsibility is to produce reliable, traceable test evidence while supporting safe and repeatable test operations.

Relevant capabilities

- Python development for automation and analysis.
- Real-time simulation or HIL systems.
- Controls and state-machine implementation.
- Instrumentation and synchronized acquisition.
- Electrical/software troubleshooting.
- Data processing, visualization, and validation.
- Git, testing, documentation, and configuration control.

Representative accomplishment

Use one concise STAR example with a quantified result: problem, personal action, validation, measured impact, and lasting mechanism.

Why Blue Origin and New Glenn

Connect mission, reusable launch systems, safe human-capable architecture, reliable test infrastructure, and your technical growth.

First-year contribution

Aim to become productive in the team's DAQ and control architecture, own scoped test-system improvements, strengthen automated validation/documentation, and contribute dependable troubleshooting and commissioning support.

22 Seven-Day Preparation Plan

Day	Focus
1	Map job responsibilities and qualifications to real examples. Identify weakest two areas. Draft introduction, "Why Blue Origin," and "Why New Glenn."
2	Review sensors, signal conditioning, ADC resolution, sampling, aliasing, filtering, uncertainty, calibration, noise, and grounding. Practice two system designs aloud.

3	Review state machines, PID, interlocks, permissives, watchdogs, fail-safe design, aborts, commissioning, and fault insertion. Draw a pressure-test state machine from memory.
4	Review Python architecture, hardware abstraction, threading/queues, logging, exceptions, unit/integration testing, persistence, Git, and configuration management.
5	Review time-series processing, alignment, filtering, outliers, test metrics, traceability, DAQmx terms, hardware clocks/triggers, and LabVIEW architecture concepts.
6	Write eight STAR stories. Map each to two or three principles. Practice two-minute and five-minute versions with numerical results and lessons.
7	Run a 90-minute mock panel: intro, resume deep dive, DAQ, controls, troubleshooting, Python architecture, behavioral questions, your questions, and closing statement.

23 Final Interview Checklist

23.1 Technical

- Explain a complete sensor-to-database chain.
- Calculate ADC resolution conceptually.
- Explain practical sampling-rate selection and anti-alias filtering.
- Compare thermocouples and RTDs.
- Explain load-cell bridge measurements and shunt calibration.
- Troubleshoot grounding, shielding, and pump-correlated noise.
- Design a test state machine with interlocks, watchdogs, and safe state.
- Describe robust Python DAQ architecture.
- Explain hardware versus software timing.
- Describe commissioning, checkout, and fault injection.
- Explain traceability, calibration, uncertainty, and data quality flags.

23.2 Behavioral

- Ownership story.
- Safety/high-standards story.
- Root-cause and operational-excellence story.
- Difficult technical problem.
- Failure and lesson learned.
- Conflict/disagreement.
- Tight deadline without compromising safety.
- Cross-functional collaboration.
- Resourcefulness.
- Mission motivation.

23.3 Presentation

- 60–90 second introduction.
- Two-minute STAR versions with quantified outcomes.
- Clear distinction between “I” and “we.”
- Five thoughtful questions.
- Concise closing statement.

24 Recommended Closing Statement

Closing statement

“Thank you for the discussion. This role strongly matches the work I want to continue developing–real-time systems, data acquisition, controls, test automation, and engineering analysis. I am particularly interested in owning the complete path from instrumented hardware to trusted test results. Based on what I have learned, I believe my background in [your strongest two areas] would let me contribute to the team while continuing to grow in [relevant area]. I am excited about the opportunity to support safe and repeatable testing for New Glenn.”